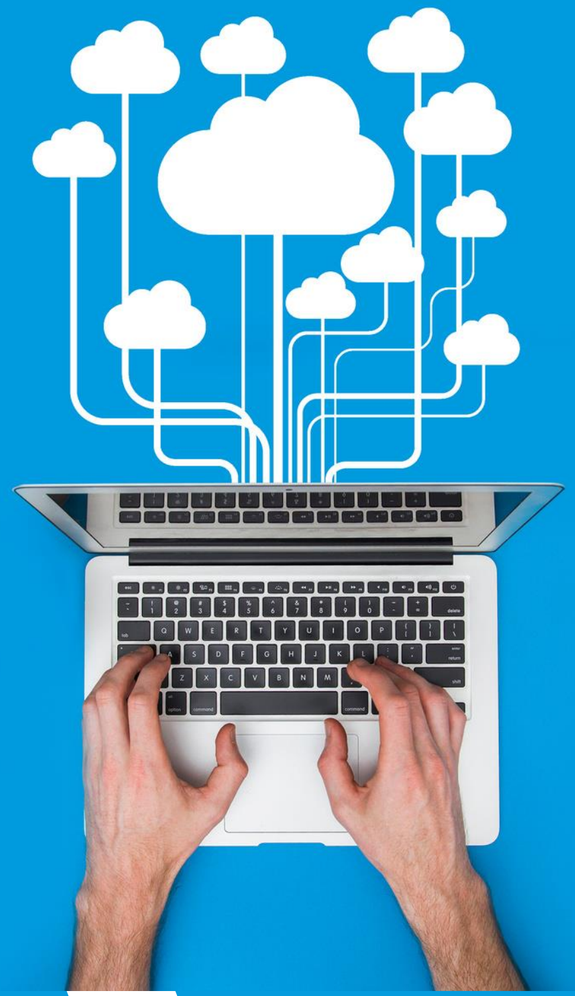
PYTHON PROGRAMMING



21 Days Challenge

Day-14

Day-14

PYTHON OOP : INHERITANCE & ACCESS MODIFIERS





Abdullah Al Zubaer

Instructor,
Artificial Intelligence & Data Science
Gobeshona Learning Academy.
B.Sc. in Internet of Things and Robotics Engineering
Gazipur Digital University, Gazipur, Bangladesh

Table Of Content

- Inheritance in Python
- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance
- Super Function
- Access Modifiers

Inheritance

Inheritance allows a class to inherit attributes and methods from another class.

```
class Animal:    # Parent
    def speak(self):
        print("Animals make sounds")

class Dog(Animal):    # Child inherits Animal
    def bark(self):
        print("Dog barks")
```

```
dog = Dog()
dog.speak()
dog.bark()
```

Single Inheritance

One parent → one child

```
class Animal:    # Parent
    def speak(self):
        print("Animals make sounds")

class Dog(Animal):    # Child inherits Animal
    def bark(self):
        print("Dog barks")
```

```
dog = Dog()
dog.speak()
dog.bark()
```

Multiple Inheritance

One child → Multiple parents

```
class Father:
    def skill1(self):
        print("Father: Driving")

class Mother:
    def skill2(self):
        print("Mother: Cooking")
```

```
class Child(Father, Mother):
    def hobby(self):
        print("Child: Gaming")

c = Child()
c.skill1()
c.skill2()
c.hobby()
```

Multilevel Inheritance

Grandparent → Parent → Child

```
class GrandParent:
    def house(self):
        print("Grandparent: Owns a house")

class Parent(GrandParent):
    def car(self):
        print("Parent: Owns a car")
```

```
class Child(Parent):
    def bike(self):
        print("Child: Owns a bike")

c = Child()
c.house()
c.car()
c.bike()
```

Hierarchical Inheritance

One parent → many children

```
class Parent:
    def asset(self):
        print("Parent: Has property")

class Son(Parent):
    def son_skill(self):
        print("Son: Playing football")

class Daughter(Parent):
    def daughter_skill(self):
        print("Daughter: Painting")
```

```
s = Son()
d = Daughter()
s.asset()
d.asset()
```

Hybrid Inheritance

Combination of two or more types

```
class A:
    def showA(self):
        print("A")

class B(A):
    def showB(self):
        print("B")

class C(A):
    def showC(self):
        print("C")
```

```
class D(B, C):
 def showD(self):
 print("D")

d = D()
d.showA()
d.showB()
d.showC()
d.showD()
```

Super Function

Used to call parent class methods in child class.

```
class Parent:
    def greet(self):
        print("Hello from Parent")

class Child(Parent):
    def greet(self):
        super().greet()  # call parent method
        print("Hello from Child")
```

```
c = Child()
c.greet()
```

Access Modifiers in Python

access modifiers define how variables and methods of a class can be accessed.

- Public Members
- Protected Members
- Private Members

Public Members

Accessible anywhere inside or outside the class.

```
class Student:
    def __init__(self, name):
        self.name = name  # public attribute

    def show(self):  # public method
        print("Name:", self.name)
```

```
s1 = Student("Alice")
print(s1.name)
s1.show()
```

Protected Members

- Marked with single underscore _var.
- Should be accessed only within the class and subclasses

```
class Person:
    def __init__(self, name, age):
        self._age = age   # protected attribute
        self.name = name

class Student(Person):
    def show(self):
        print("Name:", self.name)
        print("Age:", self._age)
```

```
s1 = Student("Bob", 21)
s1.show()
print(s1._age)
```

Private Members

- Marked with double underscore __var.
- Not directly accessible outside the class.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance   # private attribute

    def show_balance(self):
        print("Balance:", self.__balance)
```

```
acc = BankAccount(5000)
acc.show_balance()
# print(acc.__balance)
print(acc._BankAccount__balance)
```

Summery

- Single → 1 Parent → 1 Child
- Multiple → Many Parents → 1 Child
- Multilevel → Grandparent → Parent → Child
- Hierarchical → 1 Parent → Many Children
- Hybrid → Mix of above
- super() → Calls Parent's method
- Public = Road (anyone can use)
- Protected = House (family & guests)
- Private = Bedroom (only owner)



ASSIGNMENT

Create a Person class with **public**, **protected**, and **private** attributes; inherit it in Employee and Manager classes with extra attributes/methods; demonstrate inheritance and access modifiers by creating objects and trying to access attributes.

QUESTION SESSION

THANK YOU

https://gobeshona.com.bd/